

Chapter 8

Statement-Level Control Structures

Türkçe Anlatım · Terimler İngilizce

Konu Anlatımı + Review Questions + Problem Set + Programming Exercises

Gebze Technical University · Spring 2026

İçindekiler

1 Bölüme Genel Bakış	2
2 Selection Statements	4
2.1 Two-Way Selection Statements	4
2.1.1 Motivasyon	4
2.1.2 Control Expression	5
2.1.3 Clause Form ve Python Indentation	6
2.1.4 Dangling Else Problemi	6
2.1.5 Selector Expressions	7
2.2 Multiple-Selection Statements	8
2.2.1 Motivasyon ve Design Issues	8
2.2.2 C switch ve Fall-Through	8
2.2.3 C# switch ve Güvenilirlik	10
2.2.4 Ruby case, Python elif ve Scheme COND	10
2.2.5 Multiple Selection Implementation	11
3 Iterative Statements	11
3.1 Counter-Controlled Loops	12
3.2 Logically Controlled Loops	14
3.3 User-Located Loop Control Mechanisms	15
3.4 Iteration Based on Data Structures	17
4 Unconditional Branching	18
5 Guarded Commands	19
6 Conclusions	20
7 Bölüm Özeti: Tüm Kavramlar Bir Bakışta	21
8 Sınav İçin Kritik Noktalar	21
9 Review Questions — Soru Çözümleri	23
10 Problem Set — Soru Çözümleri	28
11 Programming Exercises — Ek Çözümler	31

1 Bölüme Genel Bakış

Bu chapter, program içindeki execution sequence'in statement seviyesinde nasıl yönetildiğini inceler. Chapter 7'de expression düzeyinde operator precedence ve associativity ile oluşan control flow ele alınmıştı. Burada ise statement'lar arasında seçim, tekrar, erken çıkış ve koşullu komut yapıları tartışılır.

Tanım: Control Statement ve Control Structure

Control statement, programın normal sequential execution sırasını seçme veya tekrar etme yoluyla değiştiren statement'tır. **Control structure** ise bir control statement ile onun execution'ını yönettiği statement grubunun tamamıdır.

Tarihsel Not: Fortran ve IBM 704

İlk başarılı yüksek seviyeli dillerden Fortran'daki control statements, büyük ölçüde IBM 704 machine instructions ile ilişkilidir. Bu yüzden erken control statement tasarımları dil tasarımından çok makine mimarisinin izlerini taşır.

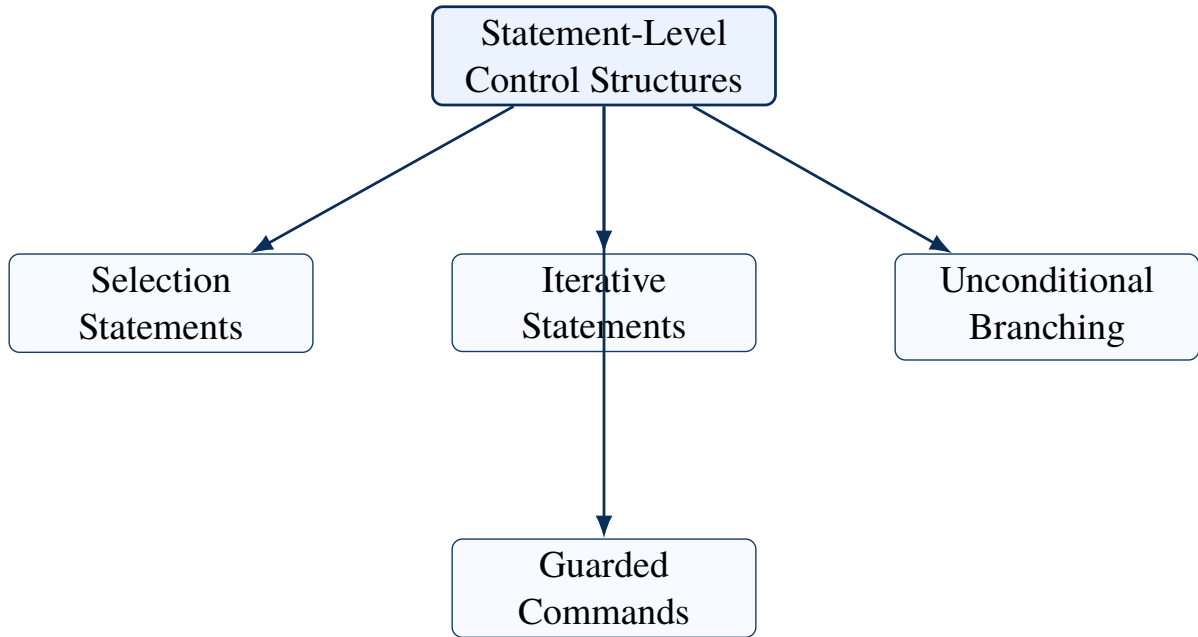
Sezgisel Bakış

Bir imperative program yalnızca assignment statement'lardan oluşsaydı, her adım tek tek yazılmak zorunda kalırdı. Selection statements karar vermeyi, iterative statements ise tekrar eden işleri kısa ve okunabilir biçimde ifade etmeyi sağlar.

Bu chapter'ı anlamanın en iyi yolu, control statement'ları yalnızca syntax olarak ezberlemek değil, her birinin hangi **control-flow problem** için tasarlandığını görmektir. Örneğin 'if' bir karar noktasını, 'while' bir koşul doğru kaldığı sürece tekrarı, 'for' sayaç veya sequence tabanlı tekrarı, 'break' ise normal loop condition dışında oluşan erken çıkış ihtiyacını ifade eder. Dil tasarımındaki temel soru şudur: Bu ihtiyaçlar kaç farklı statement formuyla ve ne kadar güvenli biçimde karşılanmalıdır?

Bu Bölümde Sürekli Sorulacak Tasarım Sorusu

Bir control structure yazmayı kolaylaştırıyorsa ama okumayı zorlaştırıyorsa, bu iyi bir dil özelliği midir? Sebesta'nın bu chapter'daki ana yaklaşımı, her yapıyı **writability**, **readability**, **reliability**, **simplicity** ve bazen **efficiency** açısından tartmaktır.



Tanım: Böhm and Jacopini Sonucu

Böhm and Jacopini, flowchart ile ifade edilebilen algoritmaların yalnızca sequence, two-way selection ve pretest logical loop kullanılarak kodlanabileceğini göstermiştir. Bu sonuç, *goto*'nun teorik olarak zorunlu olmadığını belirtir.

Bu sonucun sınav açısından iki farklı yorumu vardır. Dar anlamda yorum, “her algoritma bu üç temel yapı ile ifade edilebilir” der. Geniş ama yanlış yorum ise “o halde diğer bütün control statements gereksizdir” sonucuna atlar. Kitabın vurgusu tam burada önemlidir: theoretical minimality, practical language design için tek ölçüt değildir. Bir ‘for’ loop, teorik olarak ‘while’ ile yazılabilir; ama sayaç kontrollü bir tekrar varsa ‘for’ hem niyeti daha doğrudan gösterir hem de programcı hatasını azaltabilir.

Sınav Uyarısı

“Teorik olarak yeterli” demek, “pratikte en iyi tasarım” demek değildir. Modern dillerde ‘switch’, ‘for’, ‘foreach’, ‘break’, ‘continue’ gibi ek yapılar writability ve readability için bulunur.

Tarihsel Arka Plan: Structured Programming

1960’ların sonu ve 1970’lerde control structures üzerine yoğun tartışmalar yapılmasının nedeni, büyük programların ‘goto’ ile hızla okunamaz hale gelmesiydi. Structured programming yaklaşımı, program text order ile execution order arasındaki ilişkiyi güçlendirmeyi hedefledi. Böylece programın doğruluğu hakkında akıl yürütmek daha mümkün hale geldi.

Tasarım Ölçütü	Ne Anlama Gelir?	Control Structure Etkisi	Örnek
Readability	Kodun niyetinin kolay anlaşılması	Çok fazla form veya unrestricted branch okumayı zorlaştırır	'goto' yerine 'break'
Writability	Doğal programlama kalıplarını kısa yazmak	Özel loop ve selection formları yazmayı kolaylaştırır	'for', 'foreach'
Reliability	Hata olasılığını azaltma	Boolean-only conditions veya no fall-through güvenliği artırır	Java 'if', C# 'switch'
Simplicity	Dilin küçük ve düzenli kalması	Az sayıda construct öğrenmeyi kolaylaştırır	Minimal structured set

Kavram	Amaç	Trade-off
Minimal structures	Algoritmaları küçük bir temel setle ifade etmek	Simplicity yüksek, writability düşük olabilir
Rich control statements	Yaygın programlama kalıplarını doğrudan ifade etmek	Writability artar, dil büyür
Restricted exits	'goto' ihtiyacını güvenli hedeflerle karşılamak	Esneklik sınırlı, readability daha iyi

2 Selection Statements

Selection statement, programın iki veya daha fazla execution path arasından seçim yapmasını sağlar. Bu yapıların iki ana türü vardır: *two-way selection* ve *multiple selection*.

2.1 Two-Way Selection Statements

2.1.1 Motivasyon

Two-way selector, bir Boolean kararın sonucuna göre iki olası statement grubundan birini çalıştırır. Dil tasarımında temel soru şudur: control expression nasıl yazılmalı, 'then'/'else' parçaları nasıl sınırlandırılmalı ve nested selector durumunda ambiguity nasıl çözülmelidir?

Bir 'if' statement ilk bakışta basit görünür; fakat dil tasarımında birçok kararı saklar. 'if (x)' yazmak geçerli mi, yoksa 'x' kesinlikle Boolean mı olmalı? Tek statement body yeterli mi, yoksa her branch block olmak zorunda mı? 'else' yazılmadığında nested 'if' içinde anlam nasıl belirlenir? Bu soruların cevapları, dilin hem güvenilirliğini hem de programcının günlük yazım alışkanlıklarını etkiler.

Tanım: Two-Way Selector

Genel biçim şu fikri ifade eder: Eğer **control expression** true ise then clause, aksi durumda else clause çalıştırılır. Imperative dillerde bu çoğunlukla statement'tır; ML, F# ve Lisp ailesinde expression olarak value üretebilir.

Design Issue	Neden Önemli?	Tipik Sonuç
Control expression type	Yanlışlıkla assignment, integer veya null kullanımı hata doğurabilir	Java Boolean-only, C zero/nonzero
Clause form	Body sınırı açık değilse nested yapı okunmaz hale gelir	braces, indentation, 'end'
Nested selector semantics	'else' hangi 'if'e ait sorusu ambiguity yaratır	nearest unmatched 'if' kuralı
Expression vs statement	Selection value üretiyorsa type consistency gerekir	F#, ML selector expression

2.1.2 Control Expression

C89'da Boolean type olmadığı için arithmetic expressions control expression olarak kullanılır; zero false, nonzero true kabul edilir. C99, C++ ve Python hem arithmetic-benzeri hem Boolean ifadeleri destekleyebilir. Java, Ada ve C# gibi dillerde control expression Boolean olmalıdır.

Boolean-only yaklaşım, özellikle sınavda “reliability” argümanı ile ilişkilendirilmelidir. C'de 'if (x = 0)' syntactically geçerli olabilir çünkü assignment expression bir value üretir; programcı equality testi yazmak isterken assignment yapmış olur. Java'da bu tür hata çoğu durumda type system tarafından yakalanır. Buna karşılık C yaklaşımı daha kısa idiom'lara izin verir: 'while (c = getchar())' gibi ifadeler yazılabilir, fakat bu yazım açıklıktan çok deneyime dayanır.

Sınav Tuzağı: Truthiness ve Boolean Aynı Şey Değildir

Python ve JavaScript gibi dillerde birçok value truthy/falsy olarak yorumlanabilir. Java ve Ada gibi dillerde ise control expression gerçek Boolean olmak zorundadır. Bu fark, type checking ve error detection tartışmalarında önemlidir.

Dil	Control Expression	Clause Sınırı	Not
C	Arithmetic veya Boolean-benzeri	braces veya tek statement	zero/nonzero geleneği güvenilirliği azaltabilir
C++	Arithmetic veya bool	braces veya tek statement	C'ye göre type desteği daha zengindir
Java	boolean zorunlu	braces veya tek statement	'if (x)' geçersizdir
Ada	Boolean zorunlu	reserved words	explicit 'end if' readability sağlar
ML	if expression	expression branches	'else' zorunlu yapıdadır
Python	truth value testing	indentation	indentation semantic anlam taşır
Ruby	truthiness	'end'	'then' opsiyonel/bağlama bağlıdır
JavaScript	truthy/falsy	braces veya tek statement	coercion sınavda karıştırılabilir
F#	Boolean + expression	indentation/expression form	branch type'ları uyumlu olmalıdır

2.1.3 Clause Form ve Python Indentation

Listing 1: Python'da indentation semantic anlam taşır

```

1 if x > y:
2     x = y
3     print("case 1")
4 print("after if")

```

Python'da aynı indentation düzeyindeki statement'lar aynı block içindedir. C, C++, Java ve JavaScript'te indentation compiler için anlam taşımaz; braces veya statement sınırları önemlidir.

Python tasarımı, programcının görsel olarak gördüğü block yapısıyla interpreter'ın gördüğü block yapısını aynı hale getirir. Bu, readability için güçlü bir karardır. Ancak whitespace'in semantic olması, kopyalama, tab-space karışımı ve görünmeyen karakterler nedeniyle farklı bir hata sınıfı oluşturur. Bu yüzden Python'da layout discipline dilin parçasıdır; C-family dillerinde ise style guide konusudur.

Clause Form'u Nasıl Düşünmeli?

Clause form, yalnızca estetik bir syntax kararı değildir. Body'nin nerede başladığı ve nerede bittiği ne kadar açık olursa, nested selectors ve maintenance o kadar güvenli olur. Ada'nın 'end if' gibi explicit kapanışları uzun görünür ama büyük programlarda block eşleşmesini kolaylaştırır.

2.1.4 Dangling Else Problemi

Listing 2: Java tarzı dangling else örneği

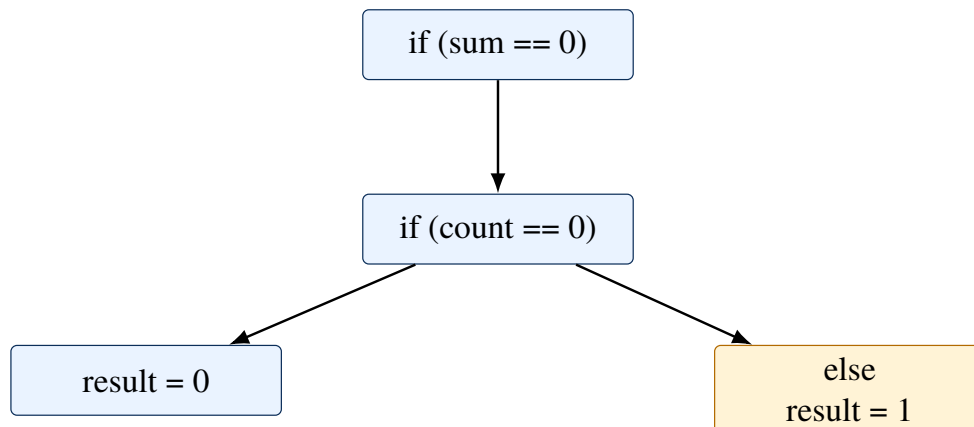
```

1 if (sum == 0)
2     if (count == 0)
3         result = 0;
4 else
5     result = 1;

```

Bu kod görsel indentation nedeniyle yanıltıcıdır. Java, C, C++ ve C# gibi dillerde 'else', en yakın unmatched 'if' ile eşleşir.

Bu problem, grammar ambiguity'nin programcı zihnindeki karşılığıdır. Programcı indentation ile dıştaki 'if'e ait bir 'else' yazdığını düşünebilir; fakat language semantics içindeki unmatched 'if'i seçer. Bu yüzden sınavda dangling else sorularında önce indentation'a değil, eşleşmemiş 'if' sayısına bakılmalıdır.



Sık Hata: Indentation'a Fazla Güvenmek

C-family dillerinde indentation sadece insan içindir. Compiler 'else' eşleşmesini syntax/static semantics kuralıyla yapar. İstenen eşleşmeyi zorlamak için braces kullanılmalıdır.

Listing 3: Braces ile istenen eşleşmeyi açık yapmak

```
1 if (sum == 0) {
2     if (count == 0)
3         result = 0;
4 } else {
5     result = 1;
6 }
```

2.1.5 Selector Expressions

Functional dillerde 'if', statement değil expression olabilir. Bu durumda seçim yapısı bir value üretir ve assignment'ın sağ tarafında kullanılabilir.

Listing 4: F# selector expression

```
1 let y =
2     if x > 0 then x
3     else 2 * x
```

Tanım: Selector Expression

Selector expression, seçilen branch'in değerini sonuç olarak döndüren selection yapısıdır. F#'ta then ve else branch'lerinin type'ları uyumlu olmalıdır; değer döndürmeyen branch 'unit' yani '()' üretebilir.

Selector expression yaklaşımı, functional programming düşüncesine daha yakındır çünkü "bir değişkene sonradan değer atamak" yerine "bir expression'ın value üretmesi" fikrini öne çıkarır. Imperative yazımda genellikle önce variable tanımlanır, sonra branch içinde assignment yapılır. Functional yazımda ise branch'lerin kendisi value üretir ve bu value doğrudan name binding için kullanılır.

Listing 5: Imperative style: branch içinde assignment

```
1 int y;
2 if (x > 0)
3     y = x;
4 else
5     y = 2 * x;
```

Listing 6: Expression style: if value üretir

```
1 let y =
2     if x > 0 then x
3     else 2 * x
```

Avantaj/Dezavantaj Analizi

Selector expression daha kısa ve referential reasoning için daha uygundur; çünkü 'y' tek noktada bind edilir. Buna karşılık branch'lerin aynı type'ta value üretmesi gerekir. Imperative selector statement daha esnektir ama branch içinde state update artıkça reasoning zorlaşır.

2.2 Multiple-Selection Statements

2.2.1 Motivasyon ve Design Issues

Bir programda iki değil, birçok path arasından seçim yapmak gerekebilir. Nested ‘if’ ile bu yazılabilir; fakat yapı büyüdükçe readability düşer. Multiple-selection statement bu seçimi tek bir syntactic yapı altında toplar.

Örneğin bir token type’a göre parser action seçmek, menu option’a göre işlem yapmak veya enum value’ya göre farklı code segment’i çalıştırmak multiple selection için doğaldır. Bu durumlarda uzun ‘if-else-if’ zinciri hem tekrar üretir hem de programcının “tek bir selector value üzerinden karar veriliyor” niyetini gizler. ‘switch’ veya ‘case’, bu niyeti daha doğrudan gösterir.

Tanım: Multiple Selection

Multiple-selection statement, bir selector expression veya bir dizi predicate üzerinden birden çok selectable segment arasından birini veya bazı dillerde birden fazlasını çalıştıran control statement’tır.

Başlıca design issues:

- Selector expression’ın formu ve type’ı nedir?
- Selectable segments tek statement mı, compound statement mı, statement sequence mı?
- Tek execution’da yalnızca bir segment mi çalışır, yoksa fall-through mümkün mü?
- Case values nasıl belirtilir?
- Unrepresented value için default davranış nedir?

2.2.2 C switch ve Fall-Through

C ‘switch’, chapter’ın en önemli reliability örneklerinden biridir. Çünkü syntax olarak temiz görünmesine rağmen semantics’i programcıdan explicit ‘break’ bekler. C tasarımında selectable segment sonunda otomatik branch yoktur; yani control, bir case label’dan sonraki statements’a akmaya devam eder.

Listing 7: C switch: break unutulursa fall-through oluşur

```

1 switch (index) {
2     case 1:
3     case 3:
4         odd + = 1;
5         sumodd + = index;
6     case 2:
7     case 4:
8         even + = 1;
9         sumeven + = index;
10    default:
11        printf("Error in switch, index = %d\n", index);
12 }
```

Bu örnekte ‘break’ olmadığı için control bir case segmentinden diğerine akar. Bu, bazen bilerek kullanılan bir flexibility sağlar; ancak çoğu durumda reliability problemidir.

Fall-through’un bilinçli kullanımı genellikle birden fazla case value’nun aynı action’a bağlanmasıdır. Örneğin ‘case 1:’ ve ‘case 3:’ boş bırakılıp aynı code segment’e düşürülebilir. Fakat segment sonunda ‘break’ unutulursa intentional grouping ile accidental fall-through dışarıdan ayırt edilmesi zor hale gelir. Bu yüzden birçok modern style guide, intentional fall-through için comment yazılmasını ister.

Listing 8: break ile tek segment execution

```

1 switch (index) {
2     case 1:
3     case 3:
4         odd + = 1;
5         sumodd + = index;
6         break;
7     case 2:
8     case 4:
9         even + = 1;
10        sumeven + = index;
11        break;
12    default:
13        printf("Error in switch, index = %d\n", index);
14 }

```

Sınav Uyarısı: C switch

C ‘switch’ statement’ında case labels normal label gibi gevşek yerleştirilebilir ve implicit branch yoktur. Bu tasarım ALGOL 68’den etkilenmiştir. Esneklik artar; yanlışlıkla fall-through reliability’yi düşürür.

C switch’i Okuma Stratejisi

Bir C ‘switch’ sorusunda her ‘case’ için üç şeyi kontrol et: case label hangi value’yu temsil ediyor, segment sonunda ‘break’ veya başka unconditional branch var mı, ‘default’ yoksa unrepresented value geldiğinde ne oluyor? Bu üç nokta çoğu sınav sorusunu çözer.

Dil	Yapı	Fall-through	Case Type	Kritik Not
C	‘switch’	Varsayılan olarak var	discrete	‘break’ gerekir
C++	‘switch’	Var	integral/enum vb.	C mirasını taşır
Java	‘switch’	Var, fakat case yerleşimi daha kısıtlı	int/enum/String vb.	boolean control değildir
Ada	‘case’	Yok	discrete	seçenekler kapsamlı olmalıdır
ML	pattern match	Yok	pattern	expression temellidir
Python	‘elif’, modern ‘match’	Yok	predicate/pattern	chapter bağlamında klasik switch yok
Ruby	‘case’	Yok	expression/range/pr	value döndürebilir
JavaScript	‘switch’	Var	strict comparison benzeri	‘break’ unutma riski
F#	pattern match	Yok	pattern	exhaustive düşünmeyi teşvik eder

2.2.3 C# switch ve Güvenilirlik

C# ‘switch’, C-family syntax’a benzer; fakat her selectable segment açık bir unconditional branch ile bitmelidir. Bu, istemsiz fall-through riskini azaltır. Ayrıca C# case values için string kullanımını destekler.

Bu tasarım, C’nin “programcı bilir” yaklaşımından daha kontrollü bir yaklaşıma geçiştir. C# hâlâ C-family yazımına yakın kalır, fakat static semantics ile yanlışlıkla bir sonraki segment’e düşmeyi engeller. Eğer programcı gerçekten bir case’ten diğerine geçmek istiyorsa ‘goto case’ gibi explicit bir form kullanmalıdır. Bu da niyeti görünür hale getirir.

Listing 9: C# switch: implicit fall-through engellenir

```

1 switch (value) {
2     case -1:
3         negatives++;
4         break;
5     case 0:
6         zeros++;
7         goto case 1;
8     case 1:
9         positives++;
10        break;
11    default:
12        Console.WriteLine("Error in switch");
13        break;
14 }
```

2.2.4 Ruby case, Python elif ve Scheme COND

Ruby ‘case’, klasik ‘switch’ten daha expression-oriented bir yapı sunar; bir value döndürebilir. Python chapter bağlamında klasik multiple-selection statement’a sahip değildir, fakat ‘elif’ zinciri Boolean predicates ile daha genel bir seçim yapısı kurar. Scheme ‘COND’ ise mathematical conditional expression fikrine yakındır: predicates sırayla test edilir ve ilk true predicate’in expression’ı value olarak döner.

Listing 10: Ruby case expression

```

1 leap = case
2     when year % 400 == 0 then true
3     when year % 100 == 0 then false
4     else year % 4 == 0
5     end
```

Listing 11: Python elif: Boolean expression tabanlı multiple selection

```

1 if count < 10:
2     bag1 = True
3 elif count < 100:
4     bag2 = True
5 elif count < 1000:
6     bag3 = True
7 else:
8     bag4 = True
```

Listing 12: Scheme COND

```

1 (COND
2   ((> x y) "x is greater than y")
3   ((< x y) "y is greater than x")
4   (ELSE "x and y are equal"))
```

Switch ile else-if Aynı Şey Değildir

‘switch’, genellikle tek bir expression’ın farklı değerlere eşitliği üzerinden çalışır. ‘else-if’ zinciri ise her branch için bağımsız Boolean expression kullanabilir. Bu yüzden daha geneldir.

Durum	Daha Uygun Yapı	Neden
Tek enum veya integer value’ya göre seçim	‘switch’ / ‘case’	Tek selector niyeti açıktır
Range veya karmaşık predicate kullanımı	‘elif’ / Ruby predicate ‘case’	Her branch farklı Boolean condition kullanabilir
Value üretmesi gereken seçim	Ruby ‘case’, F# match, Scheme ‘COND’	Selection expression gibi davranır
Accidental fall-through riskinin kabul edilemediği kod	C# switch, Ada case	Static semantics daha güvenlidir

2.2.5 Multiple Selection Implementation

Compiler, multiple selection için farklı stratejiler seçebilir. Az sayıda case varsa conditional branch zinciri yeterlidir. Çok case varsa hash table, binary search veya jump table daha verimli olabilir.

Bu implementation kısmı sınavda genellikle “compiler neden tek bir yöntem kullanmaz?” sorusuna bağlanır. Çünkü case değerlerinin sayısı, dağılımı ve type’ı farklıdır. Case values yoğun ve küçük bir aralıktaysa jump table çok hızlıdır. Fakat değerler sparse ise jump table memory israfı yapar. Ranges destekleniyorsa hash table uygun olmayabilir; binary search daha iyi bir genel çözüm olabilir.

Yöntem	Ne Zaman Uygun?	Değerlendirme
Sequential conditional branches	Case sayısı azsa	Basit ama çok case’te yavaş
Linear search table	Space daha önemliyse	Ortalama çok test gerekebilir
Hash table	Çok sayıda ayrık case varsa	Yaklaşık sabit erişim, range için uygun değil
Binary search table	Ordered case/range yapıları varsa	Dengeli ve genel çözüm
Jump table / array indexing	Case range küçük ve yoğun ise	Çok hızlı, memory harcayabilir

3 Iterative Statements

Tanım: Iterative Statement

Iterative statement, bir statement veya statement grubunun zero, one veya more times çalışmasını sağlayan yapıdır. Genellikle **loop** olarak adlandırılır.

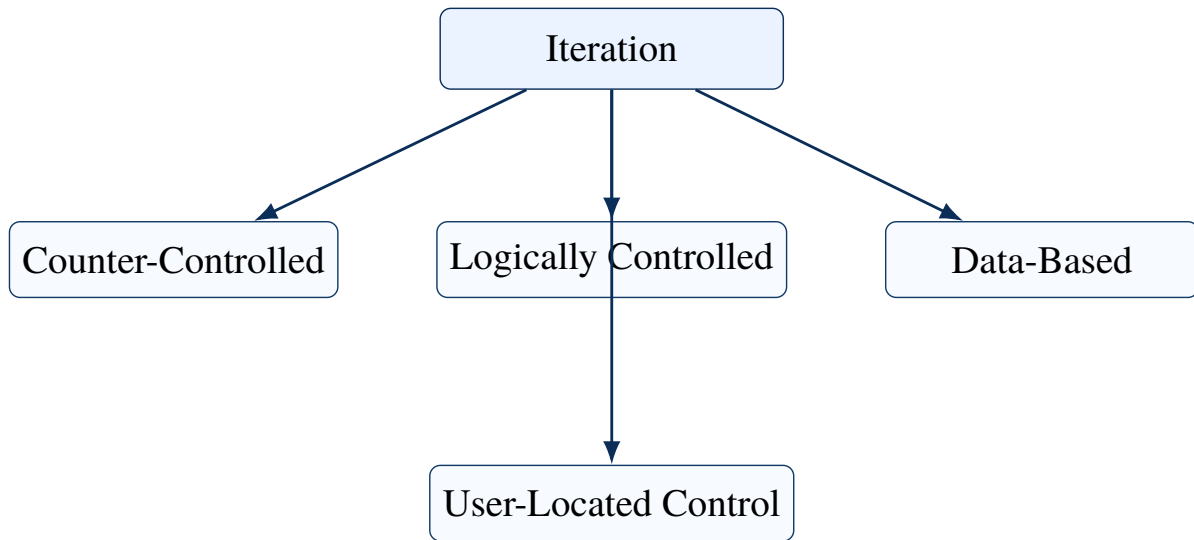
Iteration, bilgisayarın gücünü pratik hale getirir. Tekrarlama yapıları olmasaydı, her tekrar eden işlem programcı tarafından ayrı ayrı yazılmak zorunda kalırdı.

Iteration tasarımında iki eksen vardır. Birinci eksen, repetition’ın neye göre durduğudur: sayaç mı, Boolean condition mı, data structure elemanları mı? İkinci eksen, test’in nerede yapıldığıdır: body’den önce

mi, sonra mı, yoksa body'nin programcı tarafından seçilen bir noktasında mı? Bu iki eksen, chapter'daki bütün loop çeşitlerini anlamak için yeterli bir harita verir.

Loop Body ve Loop Control Ayrımı

Loop body, tekrar edilen işi temsil eder. Loop control ise bu işin kaç kez veya hangi koşula kadar yapılacağını belirler. Güvenilir loop tasarımında bu iki rol mümkün olduğunca karışmamalıdır. C 'for' gibi esnek yapılarda bu roller karışabilir; bu da hem güç hem risk üretir.



3.1 Counter-Controlled Loops

Tanım: Counter-Controlled Loop

Counter-controlled loop, bir **loop variable** ile tekrar sayısını izleyen loop'tur. Initial value, terminal value ve stepsize birlikte **loop parameters** olarak adlandırılır.

Counter-controlled loop, programcının zihnindeki “bu işi şu aralıkta şu kadar kez yap” kalıbını doğrudan ifade eder. Array traversal, fixed repetition, numeric ranges ve table generation gibi işler bu kategoriye girer. Logically controlled loop ile aynı işi yapmak mümkündür; fakat loop variable initialization, test ve update kodun farklı yerlerine dağılırsa hata olasılığı artar.

Design issues:

- Loop variable'in type ve scope'u ne olmalı?
- Loop variable veya loop parameters body içinde değiştirilebilir mi?
- Loop parameters bir kez mi, her iteration'da mı evaluate edilir?
- Loop bitince loop variable'in değeri ne olur?

Listing 13: C for statement operational fikri

```

1 for (count = 1; count <= 10; count++) {
2     process(count);
3 }
  
```

C ‘for’ statement’ı üç bölgeye ayrılır: initialization, loop test ve update. Bu üç bölge syntax içinde yan yana durduğu için sayaç kontrollü tekrarlar kompakt görünür. Ancak C’de bu bölgeler yalnızca “loop parameter” olmak zorunda değildir; herhangi expression yazılabilir. Bu yüzden C ‘for’, sadece counting loop değil, genel-purpose loop template gibi davranır.

C ‘for’ şu operational fikre yakındır:

Listing 14: C for statement operational semantics

```

1 expression_1
2 loop:
3     if expression_2 is false goto out
4     loop_body
5     expression_3
6     goto loop
7 out:

```

Listing 15: C for çok esnektir: multiple expressions

```

1 for (count1 = 0, count2 = 1.0;
2     count1 <= 10 && count2 <= 100.0;
3     sum = ++count1 + count2, count2 *= 2.5) {
4     /* Empty body: all work is in the control expressions */
5 }

```

C for Trade-off

C ‘for’ son derece flexible’dır: açık loop variable yoktur, expressions çoklu olabilir, body içinde değişkenler değiştirilebilir. Bu writability sağlar; fakat reliability ve readability için risklidir.

Neden C for Sınavda Önemlidir?

C ‘for’ bir dil tasarımı trade-off örneğidir. Aynı syntax hem sade counting loop hem de side effect dolu karmaşık loop yazmaya izin verir. Bu yüzden “C for daha güçlüdür” cevabı tek başına yetmez; “bu güç reliability problemine yol açabilir” kısmı da söylenmelidir.

Listing 16: Python for ve range

```

1 for count in range(0, 8, 2):
2     print(count)
3 # Prints: 0, 2, 4, 6

```

Python ‘range(stop)’, ‘range(start, stop)’ ve ‘range(start, stop, step)’ biçimlerini destekler. Stop değeri üretilen dizinin içinde yer almaz.

Python ‘for’, C ‘for’ gibi üç expression üzerinden çalışmaz. Bir iterable object’ten sırayla value alır. Bu nedenle loop variable body içinde değiştirilse bile iterator’ın sıradaki value üretme mantığı değişmez. Bu, C’ye göre daha az flexible ama daha predictable bir modeldir.

Listing 17: Loop variable değişse bile iteration source değişmez

```

1 for x in range(3):
2     print(x)
3     x = 100
4 # Prints 0, 1, 2

```

Listing 18: F# ile counting loop simulation

```

1 let rec forLoop loopBody reps =
2     if reps <= 0 then

```

```

3      ()
4      else
5          loopBody()
6          forLoop loopBody (reps - 1)

```

Functional languages için önemli nokta, loop variable'ın mutable state olarak değil, recursive call parameter'ı olarak temsil edilmesidir. Her recursive call, “bir sonraki iteration” fikrini taşır. Bu yaklaşım referential transparency ve reasoning açısından güçlüdür; ancak imperative dillerdeki ‘for’ kadar doğrudan görünmeyebilir.

Dil	Counting Loop	Loop Variable	Kritik Nokta
C/C++	‘for’	normal variable olabilir	Çok esnek, değiştirilebilir
Java	‘for’	local tanımlanabilir	control expression boolean olmalı
Ada	‘for’	loop scope içinde	güvenli ve okunabilir
Python	‘for in range’	son değeri korur	değişiklik loop control’ü etkilemez
Ruby	‘upto’, ‘times’, ‘each’	block parameter	gerçek for iterator’a çevrilir
F#	recursion / ranges	immutable yaklaşım	functional style

3.2 Logically Controlled Loops

Tanım: Pretest ve Posttest Loop

Pretest loop, loop body çalışmadan önce condition test eder; body zero times çalışabilir. **Posttest loop**, body çalıştıktan sonra condition test eder; body en az bir kez çalışır.

Pretest loop, “önce güvenlik kontrolü, sonra işlem” mantığına uygundur. Örneğin input stream bitmişse body hiç çalışmamalıdır. Posttest loop ise “işlem en az bir kez yapılmalı, sonra devam edip etmeyeceğe karar verilmeli” durumlarında doğaldır. Menü gösterme, kullanıcıdan geçerli giriş alana kadar tekrar etme veya bir sayının digit’lerini en az bir kez işleme buna örnektir.

Listing 19: Pretest logical loop

```

1 while (indat >= 0) {
2     sum += indat;
3     indat = read_int();
4 }

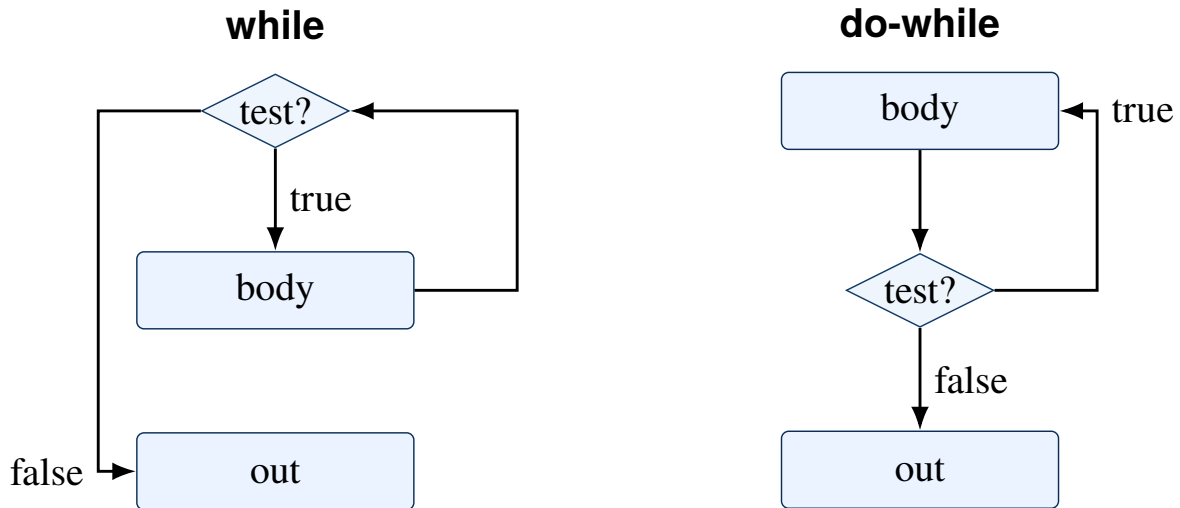
```

Listing 20: Posttest logical loop

```

1 do {
2     value /= 10;
3     digits++;
4 } while (value > 0);

```



Özellik	Pretest while	Posttest do-while
Condition zamanı	Body'den önce	Body'den sonra
Minimum execution	0	1
Risk	Condition başta yanlışsa hiç çalışmaz	Body'nin en az bir kez çalışması unutulabilir
Kullanım	Sentinel input, validation, search	Menü, en az bir kez input alma

Posttest Loop Tuzağı

Posttest loop body'yi mutlaka en az bir kez çalıştırır. Eğer body içinde array access, input parsing veya pointer dereference gibi riskli operation varsa, condition'ın body'den sonra test edildiği unutulmamalıdır.

Listing 21: F# logical loop simulation

```

1 let rec whileLoop test body =
2   if test() then
3     body()
4     whileLoop test body
5   else
6     ()

```

3.3 User-Located Loop Control Mechanisms

Bazen loop kontrolünün top veya bottom yerine body içinde belirlenmesi daha okunabilir olur. 'break', 'continue', labeled break ve Perl 'last' gibi yapılar bu ihtiyacı karşılar.

Bu mekanizmalar özellikle search ve validation kodlarında doğal görünür. Aranılan eleman bulununca loop condition'ın normal sona ermesini beklemek yerine 'break' ile çıkmak programın niyetini daha açık gösterebilir. Benzer şekilde invalid input veya skip edilmesi gereken record için 'continue', kalan body'yi gereksiz indentation içine almadan atlamayı sağlar.

Tanım: User-Located Loop Control

User-located loop control, loop çıkışının veya iteration atlamasının programcı tarafından loop body içinde seçilen bir noktaya yerleştirilmesidir.

Listing 22: Java labeled break ile outer loop'tan çıkış

```

1 outerLoop:
2 for (int row = 0; row < numRows; row++) {
3     for (int col = 0; col < numCols; col++) {
4         sum += mat[row][col];
5         if (sum > 1000.0)
6             break outerLoop;
7     }
8 }

```

Listing 23: continue ile current iteration atlanır

```

1 while (sum < 1000) {
2     getNext(&value);
3     if (value < 0)
4         continue;
5     sum += value;
6 }

```

Listing 24: break ile loop tamamen biter

```

1 while (sum < 1000) {
2     getNext(&value);
3     if (value < 0)
4         break;
5     sum += value;
6 }

```

Restricted goto Fikri

'break' ve 'continue', teknik olarak sınırlı branch davranışına sahiptir. Fakat target rastgele değildir: loop sonu veya loop control mekanizmasıdır. Bu yüzden unrestricted 'goto' kadar readability zararına yol açmaz.

break ve continue Karıştırma

'break' loop'u bitirir; 'continue' loop'u bitirmez, yalnızca current iteration'ın kalanını atlar. Nested loop içinde unlabeled 'break' sadece en küçük enclosing loop'tan çıkar. Java labeled 'break' bu yüzden önemli bir readability aracı olabilir.

Yapı	Target	Kullanım
'break'	En küçük enclosing loop/switch sonrası	Erken çıkış
'continue'	En küçük enclosing loop'un control noktası	Iteration'ın kalanını atlama
Java labeled 'break'	Belirtilen enclosing statement sonrası	Nested loop çıkışı
Perl 'last'	Loop çıkışı	'break' benzeri

3.4 Iteration Based on Data Structures

Tanım: Iterator

Iterator, bir data structure'ın elemanlarını belirli sırada tek tek üreten mekanizmadır. Iterator history-sensitive olmalıdır; yani daha önce hangi elemanı verdiğini hatırlamalıdır.

Object-oriented programming'de collection representation kullanıcıdan gizlendiği için iterator özellikle önemlidir. Kullanıcı, collection'ın iç yapısını bilmeden elemanları dolaşabilir.

Bu abstraction'ın önemi şudur: Bir collection array, linked list, tree veya hash table ile implement edilmiş olabilir. Kullanıcı yalnızca “bana sıradaki elemanı ver” contract'ını kullanır. Böylece data representation değişse bile traversal yapan client code büyük ölçüde değişmeden kalır. Bu, information hiding ve data abstraction ilkeleriyle doğrudan ilişkilidir.

Listing 25: Java enhanced for

```
1 for (String myElement : myList) {
2     System.out.println(myElement);
3 }
```

Listing 26: C# foreach

```
1 List<String> names = new List<String>();
2 names.Add("Bob");
3 names.Add("Carol");
4 names.Add("Alice");
5
6 foreach (String name in names)
7     Console.WriteLine(name);
```

Listing 27: Ruby iterators and blocks

```
1 4.times { puts "Hey!" }
2
3 list = [2, 4, 6, 8]
4 list.each { |value| puts value }
5
6 1.upto(5) { |x| print x, " " }
```

Ruby'de block, method call'a parameter gibi gönderilen anonymous code parçasıdır. 'yield', method'a bağlı block'u çalıştırır.

Ruby iterator tasarımı, loop'u bir statement olmaktan çok object'e gönderilen message gibi ele alır. '4.times' ifadesinde loop davranışı integer object'in method'u üzerinden gelir. Bu yaklaşım Ruby'nin object-oriented karakterini güçlendirir: iteration bile object protocol'ün bir parçasıdır.

Listing 28: Python generator skeleton

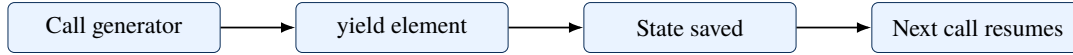
```
1 class MyStructure:
2     def traverse(self):
3         # Find the next node
4         yield node
```

Python'da 'yield' içeren method/function generator olur. Çağrılar arasında local state korunur ve execution kaldığı yerden devam eder.

Generator yaklaşımı özellikle büyük data structures veya stream-like kaynaklarda önemlidir. Tüm elemanları önce bir listede toplamak memory maliyetini artırır. Generator ise her seferinde bir eleman üretir; bu yüzden hem lazy evaluation hissi verir hem de memory kullanımını azaltır.

Iterator Avantaj/Dezavantaj Analizi

Avantaj: Representation gizlenir, traversal kodu sadeleşir, büyük veri yapılarında memory verimliliği artar. Dezavantaj: Iterator state'i ve traversal order bazen görünmez hale gelir; aynı collection üzerinde eşzamanlı değişiklik yapılırsa semantics karmaşıklaşabilir.



Dil	Yapı	Mechanism	Avantaj
Java	enhanced 'for'	'Iterable'	collection abstraction
C#	'foreach'	'IEnumerator'	generic collections ile doğal
Ruby	'each', 'times', 'upto'	block + 'yield'	expressive iterator style
Python	generator	'yield' + saved state	memory-efficient traversal
F#	sequence expressions	lazy/eager olabilir	functional pipeline

4 Unconditional Branching

Tanım: Unconditional Branching

Unconditional branch statement, execution control'ü programda belirtilen bir location'a koşulsuz aktaran statement'tır. En bilinen örnek *goto*'dur.

'goto', çok güçlüdür; diğer control structures teorik olarak 'goto' ve selection ile kurulabilir. Fakat bu güç, programın textual order'ı ile execution order'ı arasındaki ilişkiyi koparabilir. Sonuçta readability, reliability ve maintainability zarar görebilir.

Bir programı okurken doğal beklenti, statement'ların büyük ölçüde yukarıdan aşağıya çalışmasıdır. 'goto', bu beklentiyi kırar çünkü control herhangi bir label'a sıçrayabilir. Küçük bir örnekte bu anlaşılabilir; fakat büyük programda birçok label ve jump olduğunda programın gerçek execution graph'ı text'ten ayrı bir yapıya dönüşür. Bu duruma genellikle spaghetti code denir.

Tarihsel Not: Dijkstra ve Knuth

Dijkstra, 'goto' kullanımının programları karmaşıklaştırdığını savunarak structured programming tartışmasını güçlendirdi. Knuth ise bazı durumlarda 'goto'nun verimlilik veya açıklık açısından savunulabileceğini belirterek daha dengeli bir yaklaşım sundu.

Dijkstra'nın argümanı yalnızca estetik değildir; program verification ile ilgilidir. Bir loop veya selection structure'ın entry ve exit noktaları belirliyse, o yapı hakkında local reasoning yapmak mümkündür. Unrestricted 'goto', control'ün nereden gelebileceğini ve nereye gidebileceğini genişlettiği için invariant ve correctness proof yazmayı zorlaştırır.

Knuth'un daha dengeli yaklaşımı ise bütün 'goto' kullanımlarını aynı kefiye koymaz. Örneğin error cleanup, multi-level loop exit veya performance-critical low-level code gibi durumlarda 'goto' kısa ve açık görünebilir. Modern language design bu ihtiyacı genellikle restricted constructs ile karşılar: 'break', labeled break, exceptions veya structured cleanup mechanisms.

Sınav Uyarısı: goto Tartışması

‘goto’ tamamen anlamsız değildir; fakat unrestricted target seçimi program reasoning’i zorlaştırır. Modern diller, yaygın meşru ihtiyaçları ‘break’, ‘continue’, labeled break veya exception gibi daha kontrollü yapılarla karşılar.

Goto Kullanım İhtiyacı	Structured Alternatif	Neden Daha İyi?
Loop’tan erken çıkış	‘break’	Target loop sonuyla sınırlıdır
Current iteration’ı atlama	‘continue’	Loop semantics korunur
Nested loop’tan çıkış	labeled ‘break’	Target explicit ama kısıtlıdır
Error propagation	exception handling	Normal flow ile error flow ayrılır
Resource cleanup	structured cleanup / finally	Exit path’leri merkezi yönetilir

Dil	goto Durumu	Not
C	Var	Kısıtlı disiplinle kullanılmalıdır
C++	Var	Structured alternatives tercih edilir
Java	Yok	labeled break/continue vardır
Ada	Var	Strong structured constructs da vardır
ML	Yok	recursion ve expressions öne çıkar
Python	Yok	loop exits ve exceptions kullanılır
Ruby	Yok	iterator/block stili yaygındır
JavaScript	Yok	labeled statements ve break/continue vardır
F#	Yok	functional control yaygındır

5 Guarded Commands

Guarded commands, Dijkstra tarafından correctness reasoning’i kolaylaştırmak amacıyla önerilen alternatif selection ve loop yapılarıdır. Bu yapılar, deterministic dillerdeki ‘if’ ve ‘while’ formlarından farklı olarak nondeterministic choice fikrini içerir.

Tanım: Guarded Command

Guarded command, bir Boolean expression yani *guard* ile guard doğru olduğunda çalıştırılacak statement sequence’in birleşimidir. Birden fazla guard true ise seçim nondeterministic olabilir.

Guarded command fikrini anlamak için şunu düşünmek gerekir: Geleneksel ‘if-else’ yapısında branch order programcı tarafından belirlenir ve bazı durumlarda bu order aslında problemin mantığı için gereksizdir. Guarded commands, “hangi true branch seçilirse seçilsin program doğru olmalı” fikrini syntax ve semantics içine taşır. Bu yüzden program correctness hakkında reasoning yaparken branch’leri birbirinden daha bağımsız düşünebiliriz.

Listing 29: Dijkstra guarded selection syntax

```

1 if Boolean_expression -> statement
2 [] Boolean_expression -> statement
3 [] Boolean_expression -> statement
4 fi

```

Eğer birden fazla guard true ise implementation bunlardan birini seçebilir; program correctness'i hangi true guard'ın seçildiğine bağlı olmamalıdır. Hiçbir guard true değilse runtime error oluşur.

Bu nondeterminism, rastgelelik eklemek için değil, solution'ı gereksiz ayrıntıyla overspecify etmemek için önemlidir. Traditional 'if (x >= y) ... else ...' yazdığımızda eşitlik durumunda hangi variable'ın seçileceğini belirlemiş oluruz. Guarded command ise eşitlik durumunda iki seçimin de doğru olduğunu ifade edebilir.

Listing 30: Guarded command ile max

```
1 if x >= y -> max := x
2 [] y >= x -> max := y
3 fi
```

'x = y' durumunda iki guard da true olur; hangisi seçilirse seçilsin sonuç doğrudur. Bu, solution'ı gereksiz şekilde overspecify etmez.

Overspecification Nedir?

Problem açısından önemsiz olan bir detayı program text'inde zorunlu olarak seçmeye overspecification diyebiliriz. Guarded commands, özellikle birden fazla doğru action olduğunda bu gereksiz seçimi erteleyerek veya implementation'a bırakarak specification'ı daha soyut tutar.

Listing 31: Guarded loop ile dört değeri sıralama fikri

```
1 do q1 > q2 -> temp := q1; q1 := q2; q2 := temp;
2 [] q2 > q3 -> temp := q2; q2 := q3; q3 := temp;
3 [] q3 > q4 -> temp := q3; q3 := q4; q4 := temp;
4 od
```

Özellik	Traditional if/loop	Guarded Commands
Choice	Deterministic	True guard'lar arasında nondeterministic olabilir
Reasoning	Branch order önemlidir	Bağımsız guard düşüncesi güçlüdür
Runtime none case	Genellikle else/default olabilir	Selection'da error, loop'ta termination
Implementation	Basit	Daha karmaşık

Guarded Commands Sınav Tuzağı

Selection formunda hiçbir guard true değilse runtime error oluşur. Loop formunda ise bütün guards aynı anda false olduğunda loop normal şekilde biter. Bu iki davranış karıştırılmamalıdır.

CSP ve Haskell Bağlantısı

Guarded command fikri, CSP gibi concurrency mekanizmalarında ve Haskell'deki guarded function definition yapılarında etkisini göstermiştir.

6 Conclusions

Bu chapter'in ana sonucu şudur: Statement-level control structures, teorik minimum ile pratik dil tasarımı arasında denge kurar. Sequence, selection ve pretest logical loop teorik olarak yeterlidir; fakat gerçek

diller writability ve readability için counter-controlled loop, multiple selection, iterator, restricted exits gibi yapılar sunar.

Dil Tasarımı Açısından Ana Trade-off

Daha az control statement dilin simplicity'sini artırır; fakat programları daha dolaylı ve zor okunur hale getirebilir. Daha fazla control statement ise writability'yi artırır; ancak dilin öğrenilmesi ve compiler semantics'i daha karmaşık hale gelir.

7 Bölüm Özeti: Tüm Kavramlar Bir Bakışta

Kavram	Tanım	Kritik Nokta	İlgili Diller
Control structure	Statement ve kontrol ettiği body	Multiple entry readability'yi düşürür	Tüm diller
Two-way selector	İki path arasında seçim	dangling else önemli	C, Java, Python, Ruby
Selector expression	Selection value üretir	Branch type uyumu gerekir	ML, F#
Multiple selection	Çoklu path seçimi	switch ve elif aynı genellikte değildir	C, Java, Ruby, Scheme
Fall-through	Bir case'ten sonrakine akış	C'de 'break' unutulabilir	C, C++, JavaScript
Counter-controlled loop	Loop variable ile tekrar	scope/mutability design issue'dur	C, Ada, Python
Logical loop	Boolean condition ile tekrar	pretest/posttest farkı	C, Java, Ruby
User-located exit	Body içinden çıkış/atlama	restricted goto sayılabilir	Java, C, Python
Iterator	Data structure traversal	history-sensitive olmalıdır	Java, C#, Ruby, Python
Generator	'yield' ile lazy üretim	state korunur	Python
goto	Koşulsuz branch	readability ve verification riski	C, C++, C#
Guarded command	Guard + statement	nondeterministic choice	Dijkstra, CSP, Haskell etkisi

8 Sınav İçin Kritik Noktalar

- Böhm and Jacopini sonucu 'goto'nun teorik olarak zorunlu olmadığını söyler; modern constructs'ın gereksiz olduğunu söylemez.
- C-family dillerinde dangling else, en yakın unmatched 'if' ile eşleşir.
- Python'da indentation semantic'tir; Java/C/C++'ta değildir.
- F# 'if' expression value üretiyorsa then ve else type'ları uyumlu olmalıdır.
- C 'switch' fall-through sağlar; C# istemsiz fall-through'u static semantics ile engeller.
- 'elif' zinciri, 'switch'ten daha geneldir çünkü her branch bağımsız Boolean expression kullanabilir.
- Pretest loop body'yi hiç çalıştırmayabilir; posttest loop body'yi en az bir kez çalıştırır.
- C 'for' çok flexible olduğu için powerful ama potansiyel olarak unreliable'dır.

- ‘break’ loop’tan çıkar; ‘continue’ current iteration’ın kalanını atlar.
- Iterator, collection representation’ı saklayarak traversal sağlar; Python generator state’i koruyarak eleman üretir.
- Guarded commands deterministic seçim yerine nondeterministic correctness fikrini vurgular.

9 Review Questions — Soru Çözümleri

S1. Control structure nasıl tanımlanır?

Cevap

Control structure, bir control statement ile bu statement'ın execution'ını yönettiği statement veya statement sequence'in tamamıdır. Örneğin 'while' statement'ı ve onun body kısmı birlikte bir iteration control structure oluşturur.

S2. Böhm and Jacopini flowchart'lar hakkında ne göstermiştir?

Cevap

Flowchart ile ifade edilebilen algoritmaların sequence, two-way selection ve pretest logical loop kullanılarak ifade edilebileceğini göstermişlerdir. Bu, unrestricted 'goto'nun computational expressiveness için zorunlu olmadığını kanıtlar.

S3. Block nedir?

Cevap

Block, tek bir syntactic unit gibi davranan statement collection'dır. C-family dillerinde braces ile, Python'da indentation ile, Ada/Ruby gibi dillerde reserved closing words ile belirlenebilir.

S4. Selection ve iteration yapıları için ortak design issue nedir?

Cevap

Temel ortak issue, control structure'ın multiple entries'e izin verip vermemesidir. Modern görüş, multiple entries'in flexibility kazancının readability kaybına değmediği yönündedir.

S5. Selection structures için temel design issues nelerdir?

Cevap

Control expression'ın formu ve type'ı, selectable clause'ların nasıl belirtileceği, nested selectors'ın anlamı, multiple selection'da kaç segmentin çalışabileceği, case values'ın formu ve unrepresented values'ın nasıl ele alınacağı temel issues'tur.

S6. Python compound statement tasarımında ne olağandışıdır?

Cevap

Python, block sınırlarını braces veya explicit closing reserved word ile değil indentation ile belirler. Bu yüzden indentation yalnızca style değil, program semantics'inin parçasıdır.

S7. F# selector hangi durumda else clause gerektirir?

Cevap

F# 'if' expression normal bir value döndürüyorsa 'else' clause gerekir ve then/else branch type'ları uyumlu olmalıdır. 'else' yoksa then branch yalnızca 'unit' yani '()' gibi value'suz etki üretebilir.

S8. Two-way selector nesting problemine yaygın çözümler nelerdir?

Cevap

Yaygın çözümler şunlardır: ‘else’ en yakın unmatched ‘if’ ile eşleştirilir; compound statements/braces kullanılarak eşleşme açık yapılır; Ruby/Ada gibi dillerde explicit ‘end if’/‘end’ kullanılır; ML gibi diller else-less if’e izin vermeyerek ambiguity’yi azaltır.

S9. Multiple-selection statements için design issues nelerdir?**Cevap**

Selector expression’ın type’ı, selectable segments’in biçimi, tek execution’da birden fazla segmentin çalışıp çalışamayacağı, case value specification biçimi ve selector değeri hiçbir case’e uymazsa ne olacağı temel design issues’tur.

S10. Multiple selection’da birden fazla segment execution kararında hangi trade-off vardır?**Cevap**

Trade-off reliability ile flexibility arasındadır. Fall-through flexibility sağlar; fakat unutulmuş ‘break’ gibi hatalar reliability’yi düşürür. C flexibility’yi, C# reliability’yi daha çok tercih eder.

S11. C’nin multiple-selection statement’ı neden olağandışıdır?**Cevap**

C ‘switch’, implicit branch sağlamaz; bu nedenle ‘break’ yoksa execution sonraki case segmentlerine akar. Ayrıca case labels normal labels gibi oldukça gevşek yerleştirilebilir.

S12. C switch hangi önceki dilden etkilenmiştir?**Cevap**

C ‘switch’, ALGOL 68’in multiple-selection yapısından etkilenmiştir. Özellikle selectable segment sonunda implicit branch bulunmaması bu mirasla ilişkilidir.

S13. C# switch, C switch’ten nasıl daha güvenlidir?**Cevap**

C# her selectable segment’in açık bir unconditional branch ile bitmesini ister. Böylece yanlışlıkla fall-through olmaz. Ayrıca ‘goto case’ gibi explicit mekanizmalarla intentional transfer açık hale getirilir.

S14. Iterative statements için genel design issues nelerdir?**Cevap**

Iteration’ın nasıl kontrol edileceği ve control mechanism’ın loop içinde nerede yer alacağı temel issues’tur. Kontrol logical, counting veya combined olabilir; test top, bottom veya user-located noktada yapılabilir.

S15. Counter-controlled loop design issues nelerdir?

Cevap

Loop variable'ın type/scope'u, loop variable veya parameters'ın body içinde değiştirilip değiştirilemeyeceği, parameters'ın ne zaman evaluate edileceği ve loop bitince loop variable değerinin ne olacağı temel issues'tur.

S16. Pretest ve posttest loop nedir?**Cevap**

Pretest loop condition'ı body'den önce test eder; body hiç çalışmayabilir. Posttest loop condition'ı body'den sonra test eder; body en az bir kez çalışır.

S17. C++ for ile Java for arasındaki fark nedir?**Cevap**

C++ 'for' loop control expression için arithmetic veya Boolean-benzeri ifadeleri kabul edebilir. Java'da loop control expression kesinlikle 'boolean' type olmalıdır. Java ayrıca 'goto' olmadığı için loop body'ye dışarıdan branch girişine izin vermez.

S18. C for statement hangi açıdan çok esnekler?**Cevap**

C 'for' içinde initialization, test ve update parçaları expression'dır; birden fazla expression comma operator ile yazılabilir, explicit loop variable yoktur, variables body içinde değiştirilebilir ve statement counting dışı logical loop gibi de kullanılabilir.

S19. Python range function ne yapar?**Cevap**

'range', integer sequence üretir. 'range(5)' 0'dan 4'e, 'range(2,7)' 2'den 6'ya, 'range(0,8,2)' 0,2,4,6 değerlerine karşılık gelir. Stop değeri dahil edilmez.

S20. Hangi çağdaş diller goto içermez?**Cevap**

Java, Python ve Ruby chapter bağlamında 'goto' içermeyen contemporary languages olarak verilir. Bu diller erken çıkış ihtiyacını structured veya restricted constructs ile karşılar.

S21. Logically controlled loop design issues nelerdir?**Cevap**

Başlıca iki issue vardır: test pretest mi posttest mi yapılacak ve logically controlled loop, counting loop'un özel formu mu olacak yoksa ayrı bir statement mı sunulacak?

S22. User-located loop control statements neden icat edildi?

Cevap

Loop'tan body'nin ortasında veya özel bir koşulda çıkmak yaygın bir ihtiyaçtır. User-located exits bu ihtiyacı unrestricted 'goto' yerine hedefi kısıtlı, daha okunabilir branch yapılarıyla karşılar.

S23. User-located loop control mechanisms için design issues nelerdir?**Cevap**

Exit'in condition mekanizmasını kendi içinde taşıyıp taşımayacağı ve yalnızca en küçük enclosing loop'tan mı yoksa enclosing nested loop'lardan da çıkıp çıkamayacağı temel issues'tur.

S24. Java break'in C break'e göre avantajı nedir?**Cevap**

Java labeled 'break' destekler. Böylece nested loop içinde birden fazla seviyeden tek statement ile çıkılabilir. C'de unlabeled 'break' yalnızca en küçük enclosing loop veya switch'ten çıkar.

S25. C++ break ile Java break farkı nedir?**Cevap**

C++ 'break' unlabeled'dır ve en yakın loop/switch sonrasına gider. Java 'break' hem unlabeled hem labeled olabilir; labeled form, belirtilen enclosing statement'ın sonrasına transfer sağlar.

S26. User-defined iteration control nedir?**Cevap**

User-defined iteration control, özellikle abstract data type veya collection üzerinde traversal biçiminin kullanıcı veya type implementer tarafından tanımlanmasıdır. Iterator, elemanları belirli sırada üretir ve state tutar.

S27. Scheme'de multiple selection hangi function ile yapılır?**Cevap**

Scheme'de multiple selection 'COND' special form ile yapılır. Predicates sırayla evaluate edilir; ilk true predicate'in expression'ı value olarak döner.

S28. Functional language repetition'ı nasıl gerçekleştirir?**Cevap**

Pure functional languages mutable loop variable yerine recursion kullanır. Loop body bir function olarak, tekrar sayısı veya test condition ise recursive function parameter'ı olarak ifade edilir.

S29. Ruby iterators nasıl implemented edilir?**Cevap**

Ruby'de iterators, object'e gönderilen özel method calls ve onlara eşlik eden blocks ile implemented edilir. Method, 'yield' ile block'u çağırır; 'each', 'times', 'upto' yaygın örneklerdir.

S30. Predefined data structures üzerinde açıkça çağrılabilen iterators hangi dilde öne çıkar?

Cevap

Ruby, predefined iterator methods ile öne çıkar. Arrays ve hashes üzerinde 'each', sayılar üzerinde 'times' ve 'upto' gibi methods doğrudan çağrılarak iteration yapılır.

S31. Hangi yaygın dil guarded commands tasarımından parça ödünç alır?**Cevap**

Haskell, function definitions içinde guards kullanarak guarded command fikrinden izler taşır. Ayrıca CSP concurrency mekanizmaları da guarded command semantics'inden etkilenmiştir.

10 Problem Set — Soru Çözümleri

P1. Combined counting ve logical loop gereken üç durum veriniz.

Cevap

Birincisi, en fazla 100 kayıt okunacak ama sentinel value gelirse daha erken durulacak bir input loop'tur. İkincisi, array'de en fazla 'n' eleman gezilecek ama aranan value bulunursa loop bitecektir. Üçüncüsü, iterative numerical method en fazla belirli iteration sayısı çalışacak ama error toleran-ce yeterince küçük olursa erken duracaktır. Bu örneklerde hem counter hem Boolean condition gereklidir.

P2. CLU iterator özelliğinin avantaj ve dezavantajlarını değerlendiriniz.

Cevap

CLU iterator yaklaşımının temel avantajı abstraction'dır: collection representation gizlenirken traversal düzeni type implementer tarafından sağlanır. Bu readability ve modularity artırır. Dezavantaj olarak iterator semantics'i compiler/runtime için daha karmaşıktır; ayrıca stateful traversal, yanlış tasarlanırsa beklenmeyen side effect veya ordering varsayımlarına yol açabilir.

P3. Ada control statements ile C# control statements karşılaştırması.

Cevap

Ada daha explicit ve güvenilir control structures sunar: 'end if', 'end loop', güçlü typing ve kapsamlı 'case' anlayışı readability'yi artırır. C# daha familiar C-family syntax ve zengin library/foreach desteği ile writability açısından güçlüdür. Güvenilirlik öncelikli safety-critical bağlamda Ada daha iyi; mainstream application development bağlamında C# daha pratik olabilir.

P4. Compound statements için unique closing reserved words kullanmanın artı ve eksileri.

Cevap

Artıları: 'end if', 'end loop', 'end case' gibi kapanışlar nested structures'ı okumayı kolaylaştırır ve yanlış block eşleşmelerini azaltır. Eksileri: Syntax daha verbose olur, yazma maliyeti artar ve küçük programlarda gereksiz uzun görünebilir.

P5. Python indentation yaklaşımının pros/cons analizi.

Cevap

Artıları: Okunan structure ile compiler'ın gördüğü structure aynıdır; style discipline dil tarafından zorunlu kılınır. Eksileri: Whitespace görünmezliği, mixed tabs/spaces ve kopyalama sırasında indentation bozulması hata kaynağı olabilir. Sınav açısından kritik nokta: Python'da indentation semantic'tir.

P6. ALGOL 68 tarzı ters closing words okunabilirlik sorunu.

Cevap

'case' için 'esac', 'if' için 'fi' gibi ters reserved words kısa ve simetrik; fakat karakter transposition hataları veya hızlı okuma sırasında tanınabilirlik sorunu yaratabilir. 'end case' gibi explicit kapanışlar daha uzun olsa da hangi construct'ın kapandığını daha açık gösterir.

P7. Knuth ve goto tartışmasını özetleyen sınav cevabı.**Cevap**

Dijkstra çizgisi, 'goto'nun program correctness reasoning'i ve readability'yi bozduğunu savunur. Knuth ise tüm 'goto' kullanımlarının aynı olmadığını; bazı carefully controlled durumlarda efficiency veya clarity sağlayabileceğini belirtir. Modern sonuç: unrestricted 'goto'dan kaçınılır, fakat loop exits gibi restricted branch mechanisms meşru ve faydalı kabul edilir.

P8. Knuth'un multiple-exit loop önerisinin operational semantics açıklaması.**Cevap**

Multiple-exit loop, body içinde birden fazla condition kontrol ederek farklı exit noktalarına izin verir. Operational olarak loop başlar, body statements sırayla çalışır; her exit condition true olduğunda control loop dışındaki belirli target'a transfer edilir; hiçbirisi true değilse loop continuation noktasına dönlür. Bu, unrestricted 'goto' yerine targets'ı loop yapısına bağlı tutar.

P9. Java'da sadece Boolean control expression kullanmanın lehte ve aleyhte argümanları.**Cevap**

Lehte: 'if (x = y)' gibi assignment/condition karışıklıkları ve zero/nonzero convention hataları azalır; readability artar. Aleyhte: C/C++ alışkanlığı olan kısa idiomlar yazılamaz ve bazı kodlar daha verbose olur. Java tasarımı reliability'yi flexibility'ye tercih eder.

P10. Python for-else hangi durumda kullanışlıdır?**Cevap**

Search loop'ta çok kullanışlıdır. Döngü içinde aranan eleman bulunursa 'break' yapılır; loop normal biterse 'else' çalışır ve "bulunamadı" logic'i buraya yazılır. Böylece ek flag variable gereksinimi azalır.

```

1 for item in items:
2     if item == target:
3         found = item
4         break
5 else:
6     found = None

```

P11. Posttest loop gerektiren üç durum.**Cevap**

Birincisi, kullanıcıdan en az bir kez input alınması gereken menu loop'tur. İkincisi, bir sayının digit sayısını bulurken value en az bir kez işlenmelidir. Üçüncüsü, bağlantı denemesi gibi action'ın önce yapılması, sonra başarı condition'ının test edilmesi gereken retry loop'tur.

P12. C'de loop içine control transfer izninin olası nedeni.

Cevap

C, düşük seviyeli control ve compiler simplicity hedeflerini taşıyan bir dildir. Labels ve ‘goto’ mekanizması statement seviyesinde genel bırakıldığı için loop body içine transfer de yasaklanmamıştır. Bu flexibility sağlar; fakat loop invariants ve initialization atlanabileceği için reliability açısından tehlikelidir.

11 Programming Exercises — Ek Çözümler

PE1. goto’lu integer pseudocode’u loop ile yazma

Listing 32: C/Java/C# style

```

1 k = (j + 13) / 27;
2 while (k <= 10) {
3     k = k + 1;
4     i = 3 * k - 1;
5 }

```

Listing 33: Python

```

1 k = (j + 13) // 27
2 while k <= 10:
3     k = k + 1
4     i = 3 * k - 1

```

Listing 34: Ruby

```

1 k = (j + 13) / 27
2 while k <= 10
3     k = k + 1
4     i = 3 * k - 1
5 end

```

Değerlendirme

Bu örnek için Python en az syntactic gürültüye sahiptir. C/Java/C# braces nedeniyle block’u açık gösterir. Ruby ‘end’ ile okunabilir bir kapanış sağlar. Readability açısından üçü de goto’lu versiyondan daha iyidir.

PE2. Floating-point versiyon

```

1 k = (j + 13.0) / 27.0;
2 while (k <= 10.0) {
3     k = k + 1.2;
4     i = 3.0 * k - 1.0;
5 }

```

Floating-point Loop Uyarısı

Floating-point counter ile loop yazarken precision nedeniyle termination condition dikkatli seçilmelidir. Equality tabanlı termination özellikle risklidir.

PE3. Multiple-selection rewrite

Listing 35: C/Java/C# switch

```

1 switch (k) {
2     case 1:
3     case 2:
4         j = 2 * k - 1;

```

```

5         break;
6     case 3:
7     case 5:
8         j = 3 * k + 1;
9         break;
10    case 4:
11        j = 4 * k - 1;
12        break;
13    case 6:
14    case 7:
15    case 8:
16        j = k - 2;
17        break;
18 }

```

Listing 36: Python elif

```

1 if k == 1 or k == 2:
2     j = 2 * k - 1
3 elif k == 3 or k == 5:
4     j = 3 * k + 1
5 elif k == 4:
6     j = 4 * k - 1
7 elif k == 6 or k == 7 or k == 8:
8     j = k - 2

```

Listing 37: Ruby case

```

1 case k
2 when 1, 2
3     j = 2 * k - 1
4 when 3, 5
5     j = 3 * k + 1
6 when 4
7     j = 4 * k - 1
8 when 6, 7, 8
9     j = k - 2
10 end

```

Değerlendirme

Bu örnekte Ruby ‘case’ ve C-style ‘switch’ doğaldır çünkü selection tek variable’ın discrete değerlerine bağlıdır. Python ‘elif’ doğru fakat daha tekrar içerir.

PE4. goto/break kullanmadan C segment rewrite

```

1 j = -3;
2 i = 0;
3 done = 0;
4 while (i < 3 && !done) {
5     if (j + 2 == 3 || j + 2 == 2) {
6         j--;
7     } else if (j + 2 == 0) {
8         j += 2;
9     } else {
10        j = 0;
11    }

```

```

12
13     if (j > 0) {
14         done = 1;
15     } else {
16         j = 3 - i;
17         i++;
18     }
19 }

```

PE5. All-zero row örneğini goto'suz yazma

```

1 for (i = 1; i <= n; i++) {
2     all_zero = 1;
3     for (j = 1; j <= n; j++) {
4         if (x[i][j] != 0) {
5             all_zero = 0;
6             break;
7         }
8     }
9     if (all_zero) {
10        printf("First all-zero row is: %d\n", i);
11        break;
12    }
13 }

```

Readability

Goto'lu versiyon kısa olabilir; ancak flag + structured exits kullanan versiyon, target'ın neresi olduğunu daha açık gösterir. Java labeled break ile flag ihtiyacı da azaltılabilir.

PE6. Üç sayıyı max, mid, min olarak yerleştirme

Listing 38: Nested selections

```

1 if (first >= second) {
2     if (second >= third) {
3         max = first; mid = second; min = third;
4     } else if (first >= third) {
5         max = first; mid = third; min = second;
6     } else {
7         max = third; mid = first; min = second;
8     }
9 } else {
10    if (first >= third) {
11        max = second; mid = first; min = third;
12    } else if (second >= third) {
13        max = second; mid = third; min = first;
14    } else {
15        max = third; mid = second; min = first;
16    }
17 }

```

Listing 39: Daha az nested, swap style

```

1 max = first;

```

```

2 mid = second;
3 min = third;
4
5 if (max < mid) { temp = max; max = mid; mid = temp; }
6 if (mid < min) { temp = mid; mid = min; min = temp; }
7 if (max < mid) { temp = max; max = mid; mid = temp; }

```

Karşılaştırma

Nested solution tüm cases'i doğrudan yazar ama karmaşıktır. Swap style daha kısa, daha düzenli ve hata olasılığı daha düşüktür.

PE7. PE4'ü if ve goto ile C'de yazma

```

1 j = -3;
2 i = 0;
3 loop:
4 if (i >= 3) goto out;
5
6 if (j + 2 == 3) goto dec;
7 if (j + 2 == 2) goto dec;
8 if (j + 2 == 0) goto add;
9 goto def;
10
11 dec:
12 j--;
13 goto after_select;
14
15 add:
16 j += 2;
17 goto after_select;
18
19 def:
20 j = 0;
21
22 after_select:
23 if (j > 0) goto out;
24 j = 3 - i;
25 i++;
26 goto loop;
27
28 out:

```

PE8. PE4'ü Java'da switch kullanmadan yazma

```

1 j = -3;
2 for (i = 0; i < 3; i++) {
3     int selector = j + 2;
4     if (selector == 3 || selector == 2) {
5         j--;
6     } else if (selector == 0) {
7         j += 2;
8     } else {
9         j = 0;
10    }

```

```
11
12     if (j > 0)
13         break;
14     j = 3 - i;
15 }
```

PE9. Scheme COND çağrısını C'ye çevirme

```
1  if (x > 10)
2      y = x;
3  else if (x < 5)
4      y = 2 * x;
5  else if (x == 7)
6      y = x + 10;
7  /* No else: the original COND has unspecified value if no predicate is
   true. */
```

COND Else Uyarısı

Scheme 'COND' içinde 'ELSE' yoksa hiçbir predicate true olmadığında result unspecified olabilir. C çevirisinde de bu durum için explicit 'else' yoksa 'y' eski değerinde kalabilir.